

Informatica per le Biotecnologie

Algoritmica Lezione 2

Il nostro primo algoritmo in pseudocodice

RICERCA DEL MASSIMO IN UN INSIEME

Consideriamo un insieme $R = \{r_0, r_1, \dots, r_{n-1}\}$ di *n* elementi diversi tra loro per cui è definita una relazione di ordinamento indicata con *<*, *>* e cerchiamo l'elemento massimo *m*

Valutiamo la complessità degli algoritmi di soluzione come numero di confronti $C(n)$ eseguiti, in funzione di n .

A causa dell'estrema semplicità del problema proviamo a calcolare il limite in modo **preciso** anziché **in ordine di grandezza**. Preciseremo poi come si definiscono questi ordini.

Cerchiamo un algoritmo di soluzione. Il numero di confronti eseguiti ci fornirà un limite superiore alla complessità del problema.

L'algoritmo più immediato che chiamiamo MASSIMO-ITER ha forma *iterativa*, cioè specifica la sequenza di operazioni in modo esplicito.

L'insieme è memorizzato in un vettore $A[0...n-1]$, con $n > 1$. L'algoritmo scandisce il vettore A per valori crescenti dell'indice i , memorizzando in m , al passo i -esimo, il valore massimo trovato nella porzione del vettore tra $A[0]$ e $A[i]$.

Prima di codificarlo in uno **pseudocodice** vicino al linguaggio C, vediamo come funziona su un esempio.

24 15 7 9 20 49 33 35 22 40 52 1 62 30 8 43

24 15 7 9 20 49 33 35 22 40 52 1 62 30 8 43

m

24 15 7 9 20 49 33 35 22 40 52 1 62 30 8 43

m



24 15 7 9 20 49 33 35 22 40 52 1 62 30 8 43

m



24 15 7 9 20 49 33 35 22 40 52 1 62 30 8 43

m



24 15 7 9 20 49 33 35 22 40 52 1 62 30 8 43

m



24 15 7 9 20 49 33 35 22 40 52 1 62 30 8 43

m

24 15 7 9 20 49 33 35 22 40 52 1 62 30 8 43

m



24 15 7 9 20 49 33 35 22 40 52 1 62 30 8 43

m

24 15 7 9 20 49 33 35 22 40 52 1 62 30 8 43

m •

24 15 7 9 20 49 33 35 22 40 52 1 62 30 8 43

m

24 15 7 9 20 49 33 35 22 40 52 1 62 30 8 43

m • • •

Dunque il massimo è **m = 62**

MASSIMO-ITER(A, n, m)

// Ricerca del massimo elemento

in un vettore $A[0 \dots n - 1]$ //

input A, n ; **output** m ;

$m = A[0]$;

for ($i = 1; i \leq n - 1; i++$)

if ($m < A[i]$) $m = A[i]$;

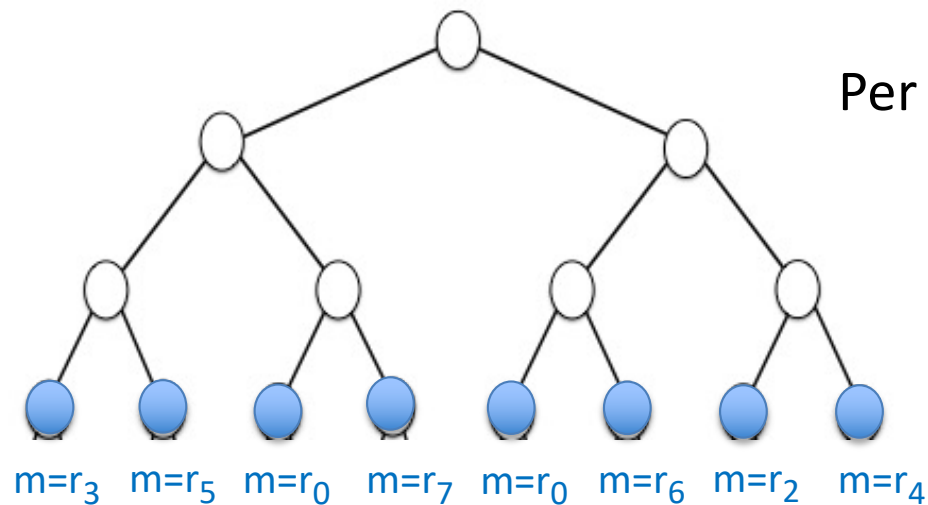
Un'analisi elementare dell'algoritmo mostra che questo è corretto e esegue esattamente $n-1$ confronti nel caso pessimo, stabilendo un **limite superiore** $C(n) = n-1$ alla complessità del problema

Valutiamo ora un **limite inferiore** per il problema.

Limite inferiore per $C(n)$

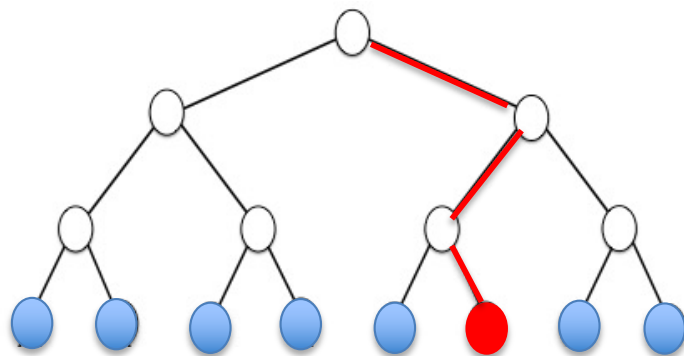
Eseguiamo ragionamenti diversi per determinare questo limite e sceglieremo il limite più alto (quindi più significativo) tra quelli trovati.

1. *Albero di decisione.* Il problema ha $s = n$ soluzioni: $m = r_0, m = r_1, \dots, m = r_{n-1}$. I confronti successivi generano un *albero binario* che contiene le soluzioni nelle foglie.



Per esempio $n = 8$

Il percorso più lungo dalla radice a una foglia determina il limite inferiore per il problema



$$n = 8 = 2^c$$

L'altezza dell'albero è

$$c = \log_2 8 = 3$$

Se n è una potenza di 2 ogni percorso può contenere al minimo $\log_2 n$ nodi di diramazione (albero bilanciato), quindi $C(n) \geq \log_2 n$

Se n non è una potenza di 2 alcuni percorsi dalla radice a una foglia contengono un elemento in meno e il logaritmo si approssima all'intero superiore. Il limite inferiore al problema è sempre $C(n) \geq \log_2 n$ relativo al percorso più lungo

2. Tutti gli elementi devono essere confrontati almeno una volta.

Poiché ogni confronto avviene tra due elementi occorrono almeno $n/2$ confronti per considerarli tutti. Dunque $C(n) \geq n/2$

3. La determinazione del massimo implica che gli altri $n-1$ elementi non sono il massimo.

Per certificare che un elemento non è il massimo occorre che risulti minore di un'altro in almeno un confronto.

Poiché da ogni confronto esce esattamente un perdente, sono necessari $n-1$ confronti per dichiarare che ci sono $n-1$ perdenti. Dunque $C(n) \geq n-1$

I tre limiti inferiori trovati

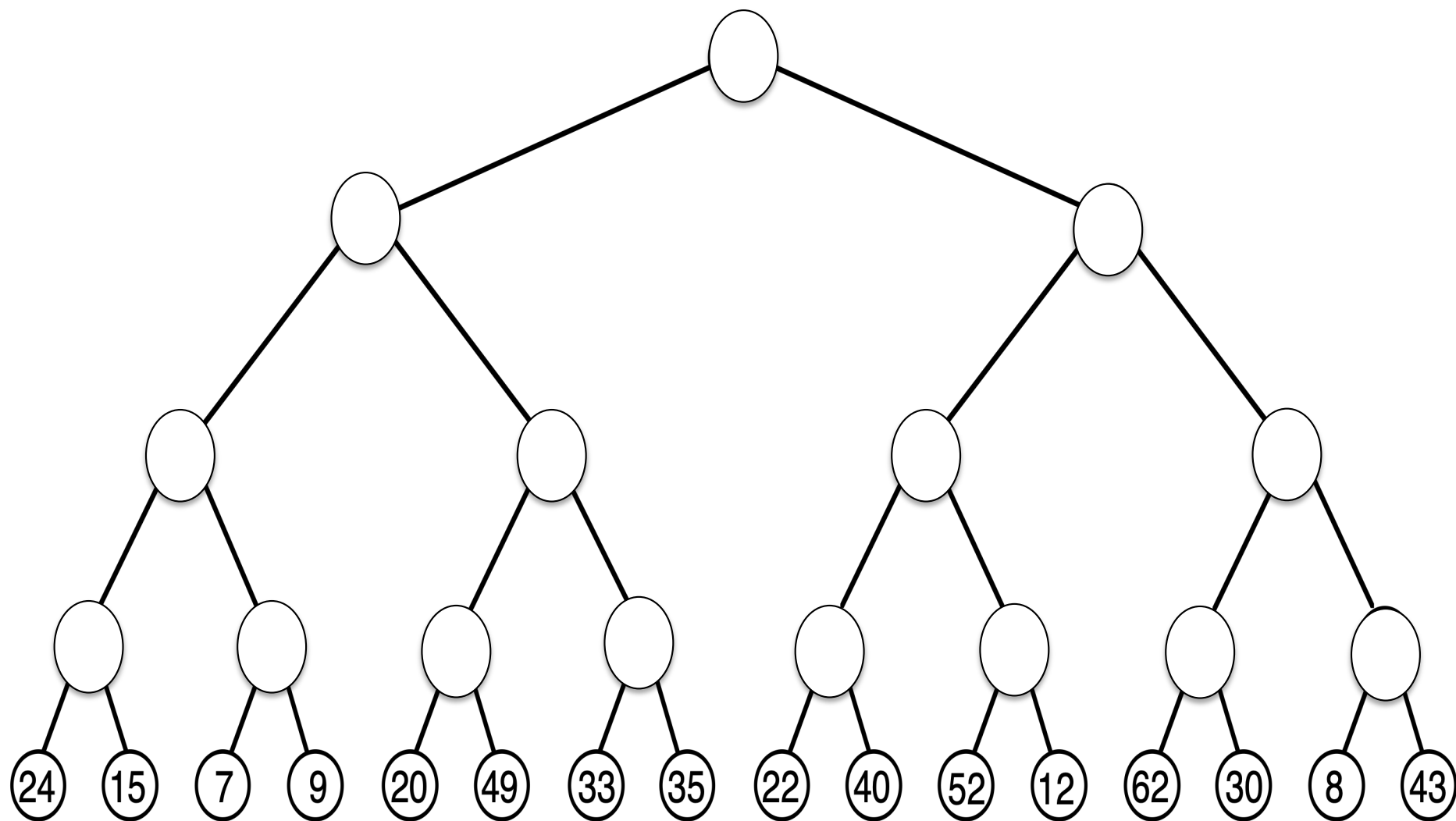
$$C(n) \geq \log_2 n, C(n) \geq n/2, C(n) \geq n-1$$

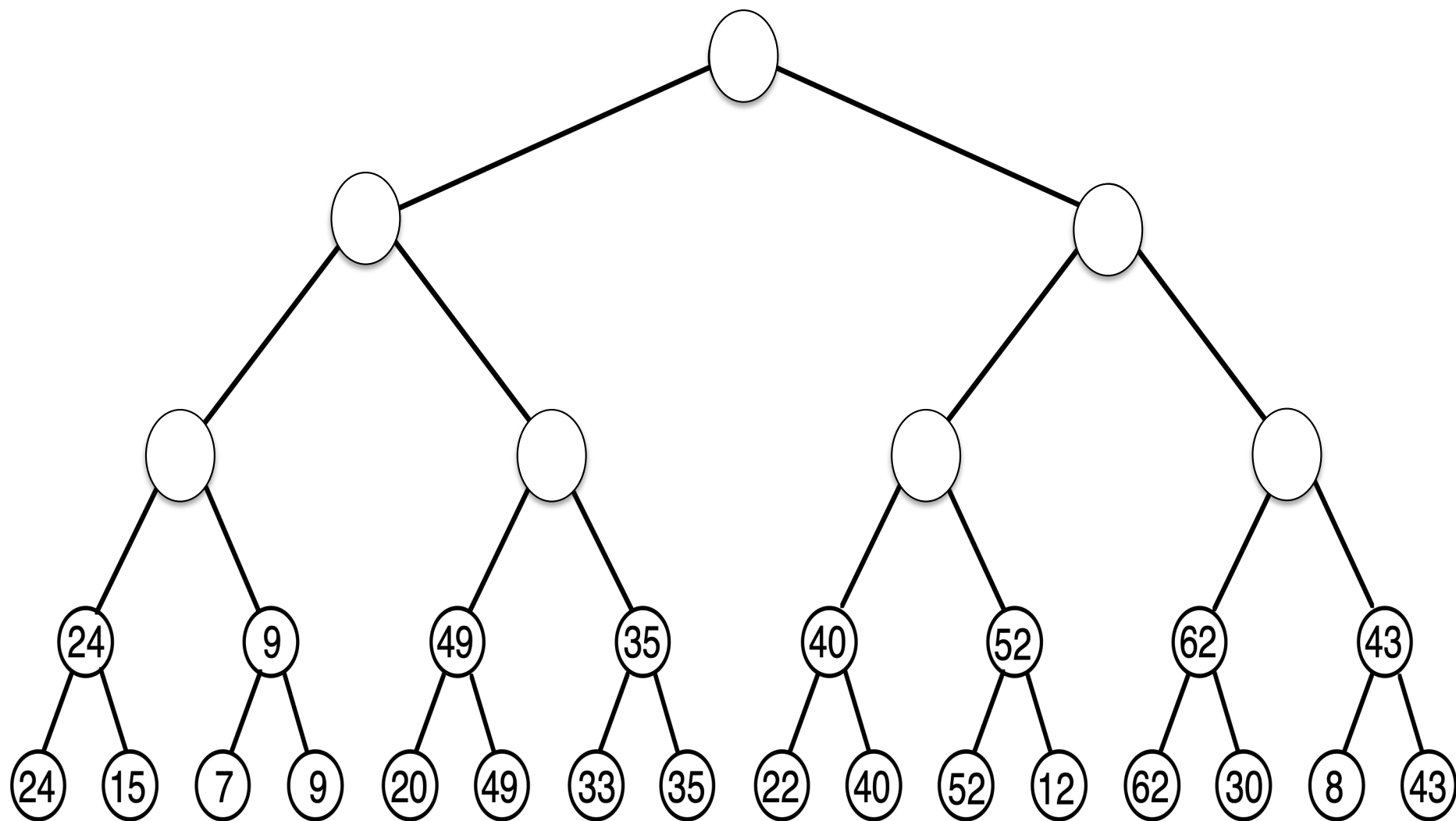
sono tutti legittimi, ma il terzo è il più grande e viene scelto come il più significativo.

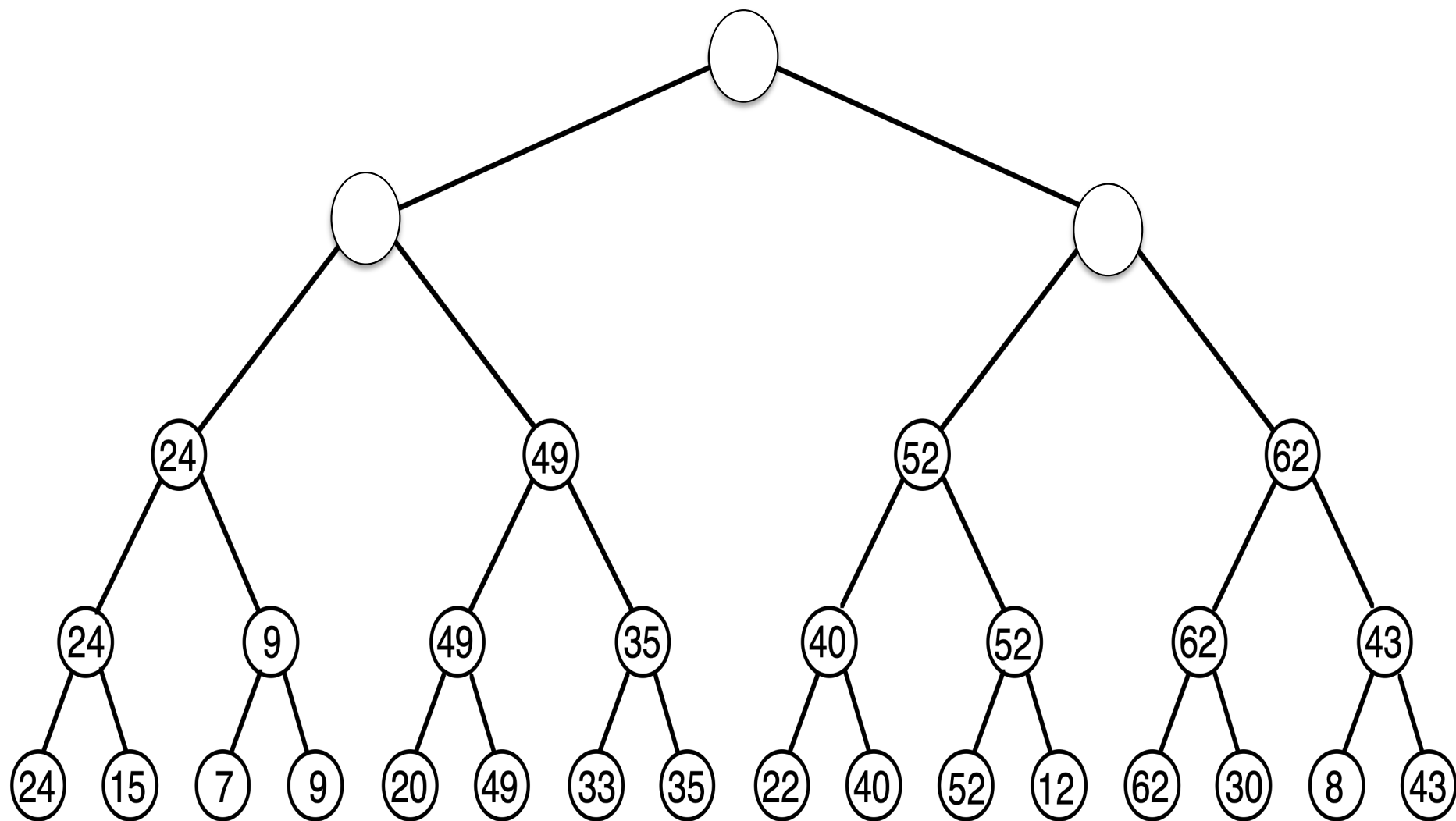
La semplicità del problema ci permette di stabilire limiti esatti. In genere potremo calcolarli solo in ordine di grandezza. Per questo problema il limite 1 è di ordine $\Omega(\log n)$, i limiti 2 e 3 sono di ordine $\Omega(n)$ quindi equivalenti.

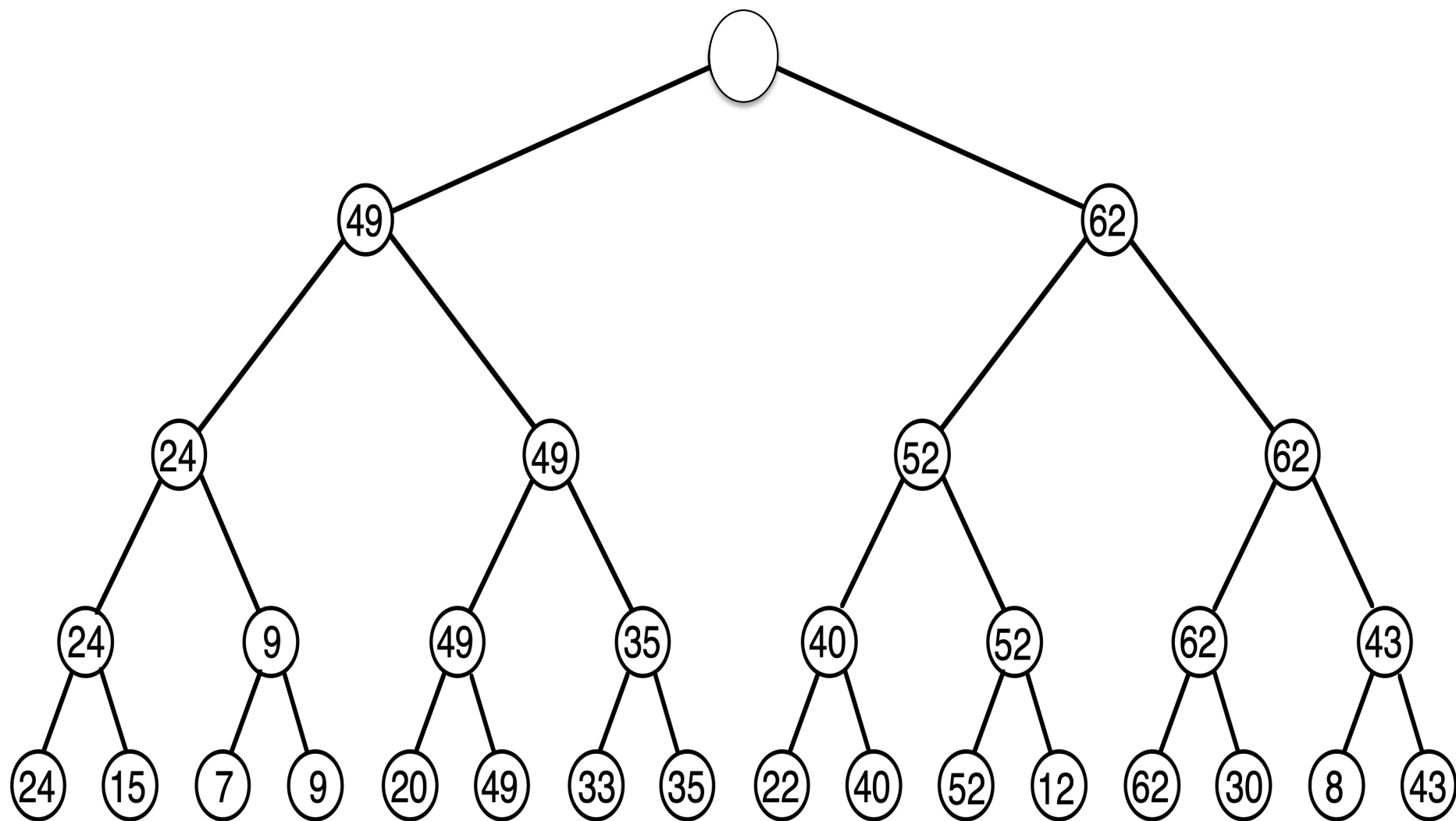
Poiché il valore $n-1$ del limite inferiore è uguale al limite superiore dell'algoritmo proposto, questo è ottimo e il problema è computazionalmente chiuso.

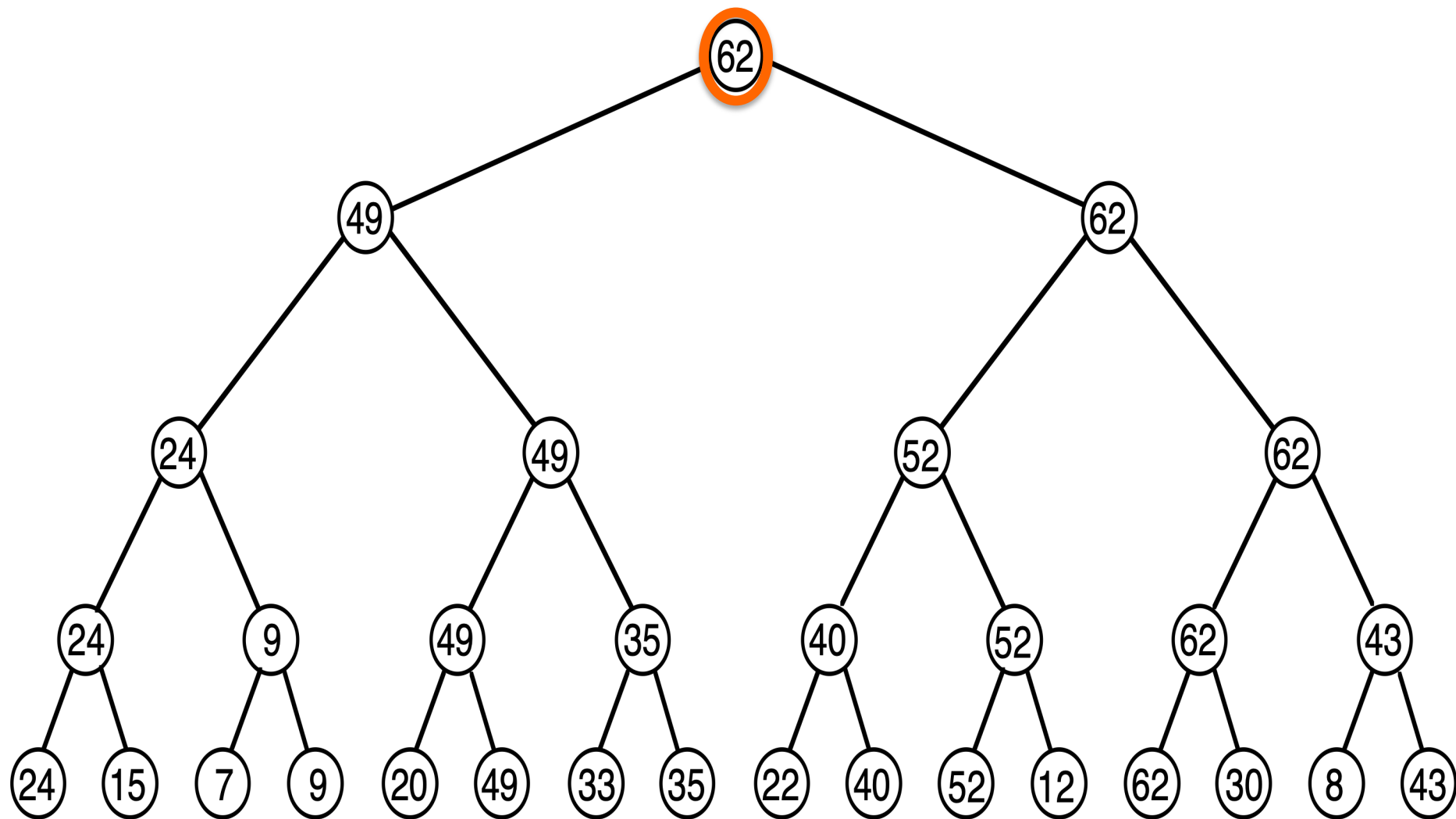
Un altro metodo per trovare il massimo di un insieme impiega la struttura di un **torneo** a eliminazione diretta









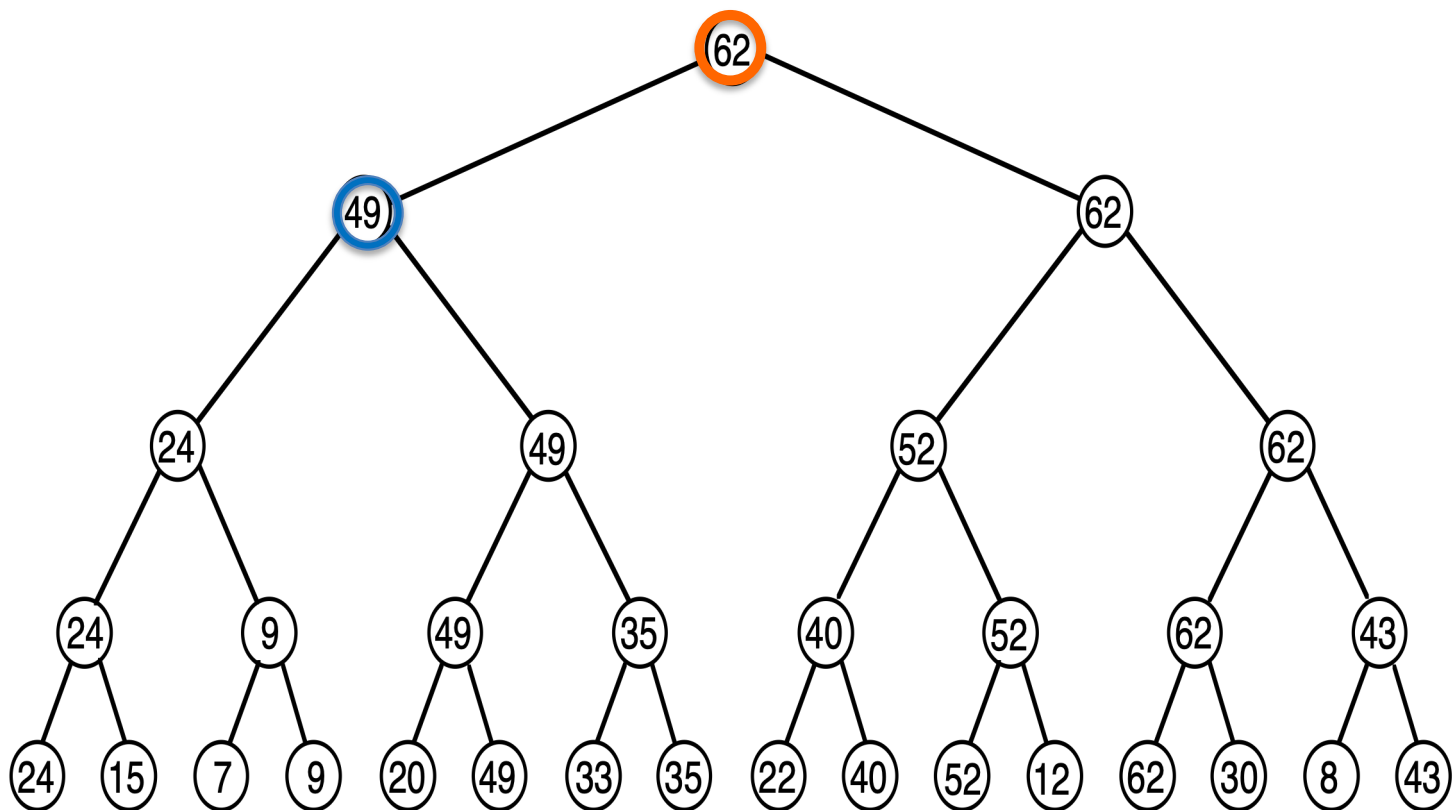


Posto $n = 2^t$, gli elementi si confrontano in coppie e restano in gara $n/2$ vincitori dopo il primo turno, $n/4$ dopo il secondo turno, fino alla finale tra 2 partecipanti. Il numero di confronti $\sum_{i=0}^{t-1} 2^i = 2^t - 1 = n - 1$

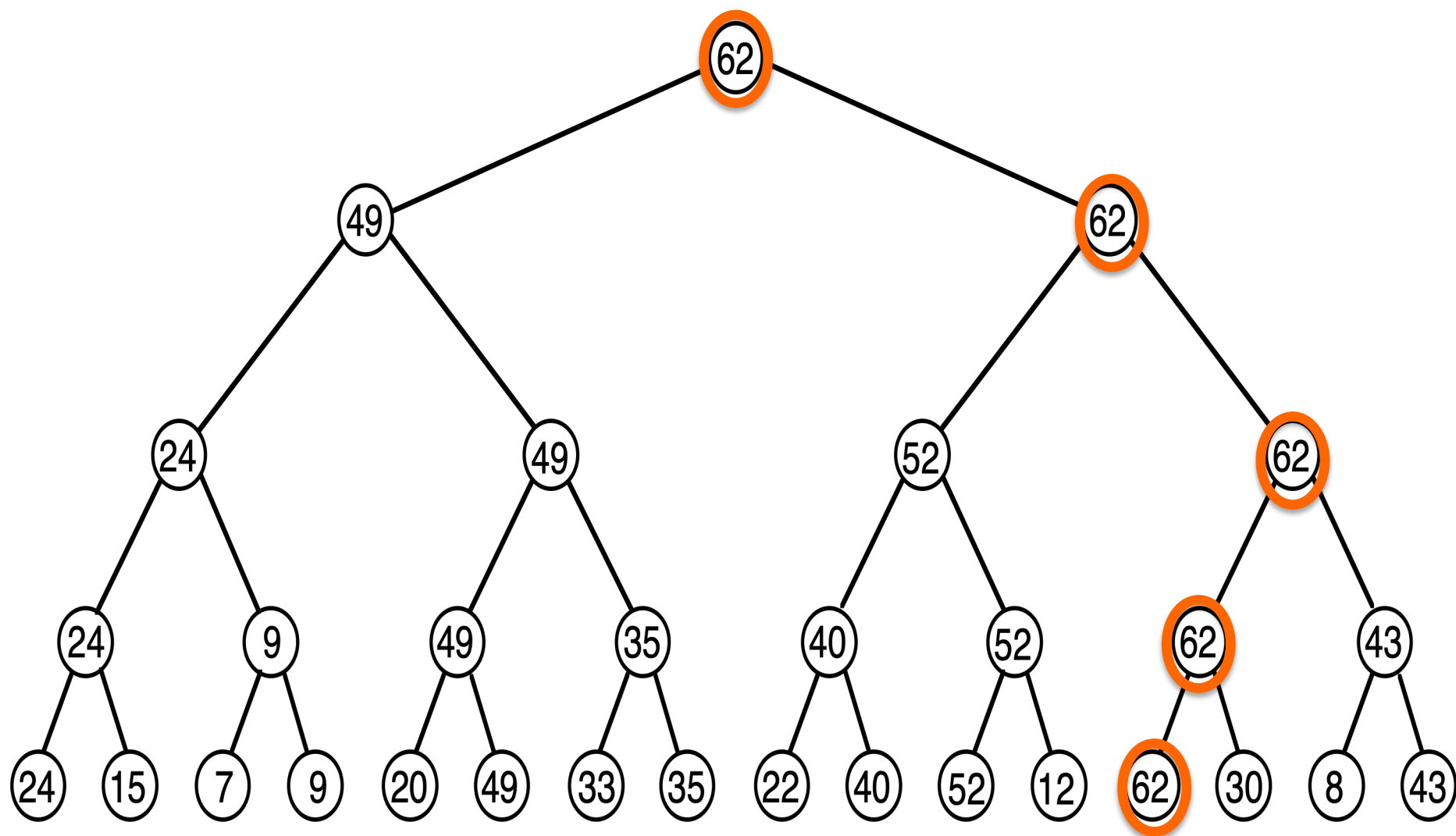
I due algoritmi sono entrambi **ottimi** anche se eseguono confronti diversi tra elementi

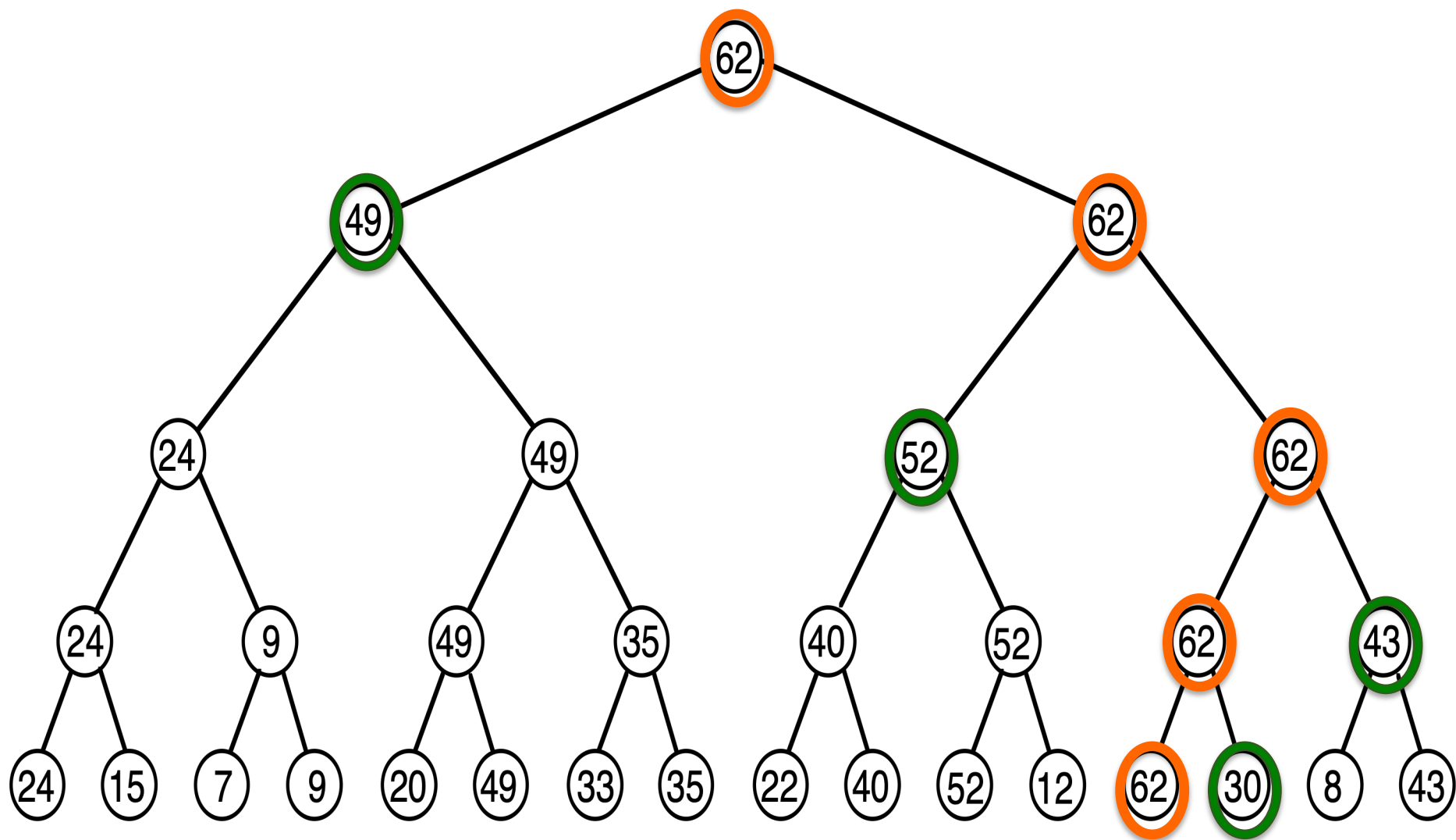
Poniamoci ora una nuova domanda sull'albero del
torneo

Lewis Carroll, St. James' Gazette, 1883



49 è entrato in finale con 62, ma è veramente il secondo?





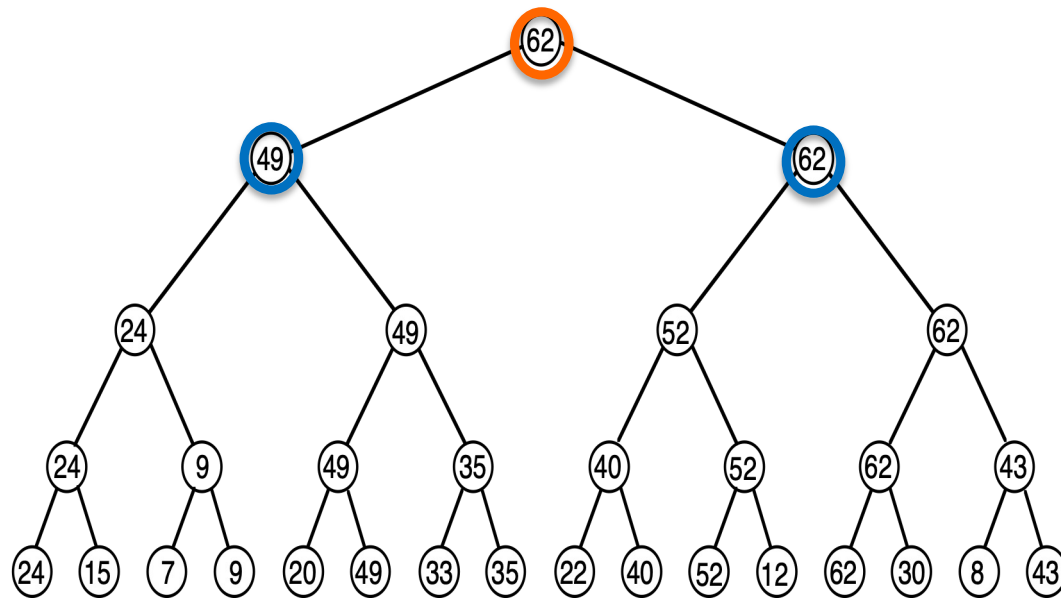
Il secondo deve essere cercato tra i candidati che hanno “perso” in un confronto con il massimo

Il numero di questi candidati è pari all'altezza $\log_2 n$ dell'albero (4 in questo caso). Per determinare il secondo classificato si può fare un altro torneo tra questi candidati con $\log_2 n - 1$ confronti, quindi $n + \log_2 n - 2$ confronti in totale (18 in questo caso)

Questo algoritmo è ottimo, ma dimostrarlo non è affatto semplice

Sull'albero del torneo abbiamo indicato **le operazioni** di confronto nell'ordine in cui vengono eseguite. Ma come si definisce “semplicemente” un algoritmo per un arbitrario numero n di elementi?

Il problema, e l'algoritmo di soluzione, possono avere una definizione **ricorsiva**.



Il problema e il suo algoritmo possono essere definiti e codificati osservando che il **campione** è il vincitore tra i **vincitori** delle due metà del tabellone.

Un procedimento ricorsivo su n dati chiama se stesso su sottoinsiemi di tali dati, finché questi sottoinsiemi raggiungono una dimensione costante (e piccola) per cui il problema è risolto in modo diretto.

L'algoritmo ricorsivo del torneo si codifica come segue

MASSIMO-RICOR(A, i, j, m)

*// Ricerca ricorsiva del massimo in un
sotto-vettore di A tra le posizioni i, j //*

input A, i, j ; **output** m ;

if ($j - i == 1$) {**if** ($A[i] < A[j]$) $m = A[j]$
 else $m = A[i]$; **return** m };

$k = \lfloor (i + j)/2 \rfloor$;

MASSIMO-RICOR(A, i, k, m_1);

MASSIMO-RICOR($A, k + 1, j, m_2$);

if ($m_1 < m_2$) $m = m_2$ **else** $m = m_1$;

Il calcolo si innesca con la “chiamata”
esterna $\text{MASSIMO-RICOR}(A, 0, n - 1, m)$
sull'intero vettore A .

Il numero $C(n)$ di confronti tra elementi è soggetto a una *relazione di ricorrenza* che si ottiene da un esame del programma:

$$C(n) = 1, \quad \text{per } n = 2;$$

$$C(n) = 2 C(n/2) + 1, \quad \text{per } n > 2.$$

La soluzione è $C(n) = n-1$ (Vedi dispense)

ORDINI DI GRANDEZZA DELLE FUNZIONI

$f(n)$

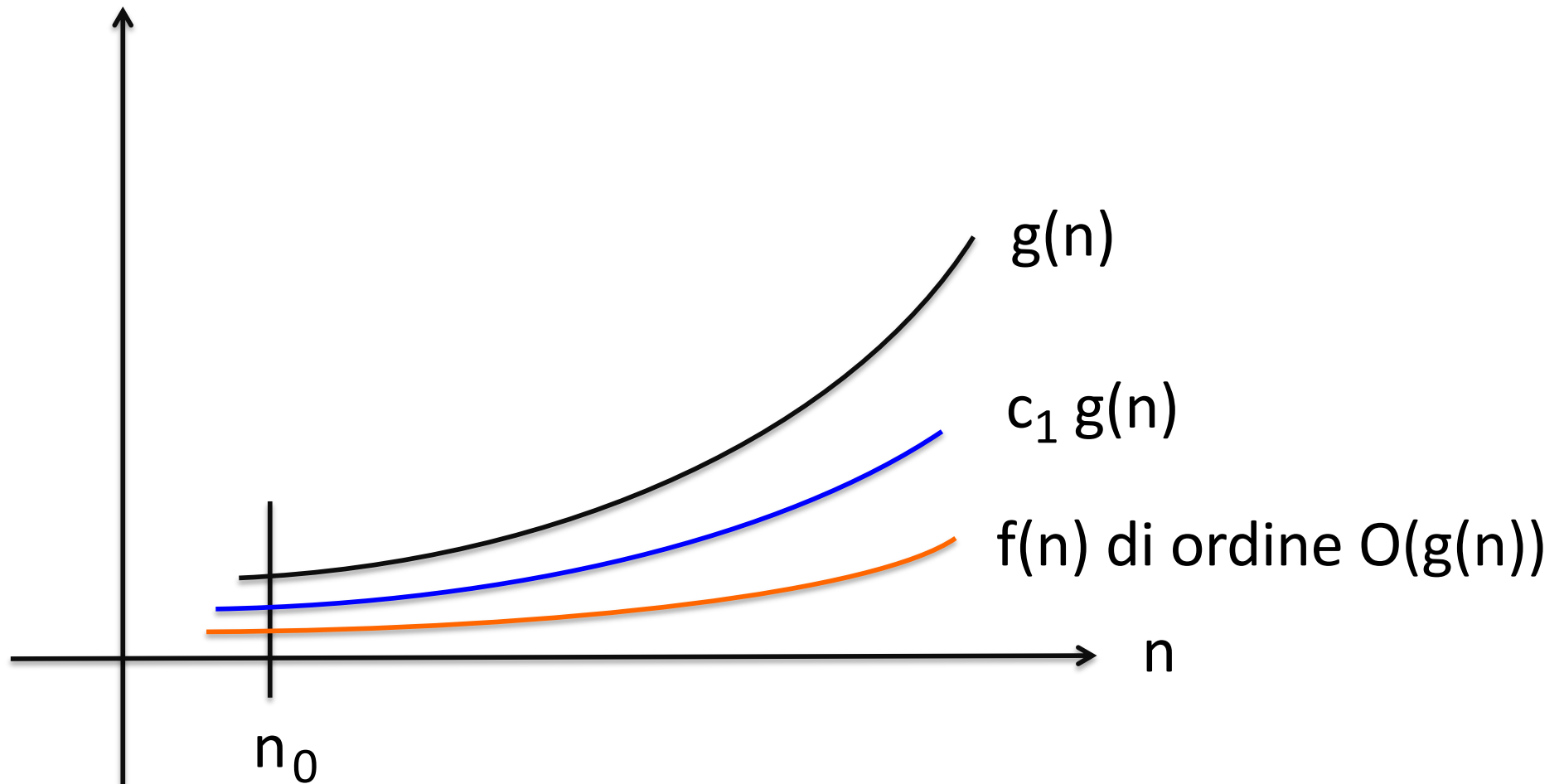
La variabile indipendente $n > 0$ indica la dimensione dei dati di un problema.

La funzione f di cui si studia l'ordine di grandezza indica la complessità di un algoritmo di risoluzione.

Notazione O

$f(n)$ è di ordine $O(g(n))$ se esistono due costanti positive c_1 , n_0 , tali che $0 \leq f(n) \leq c_1 g(n)$ per ogni $n \geq n_0$.

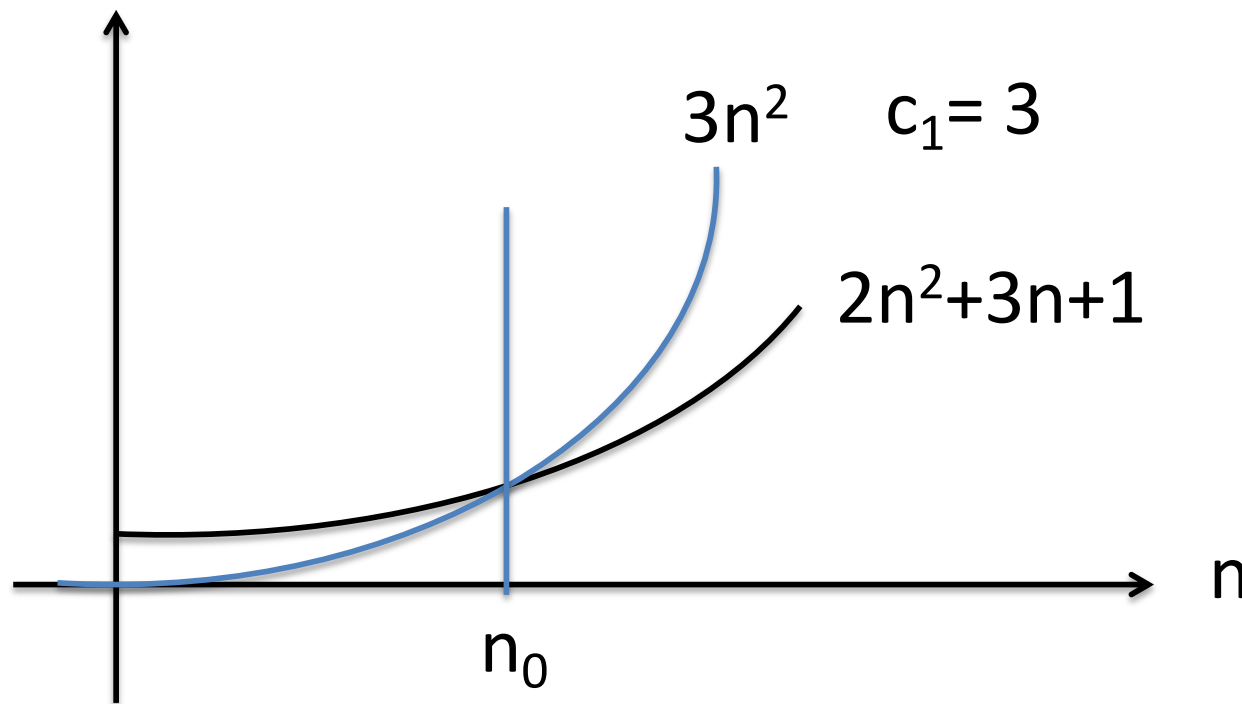
funzioni



Cioè, al crescere di n e a partire da un valore n_0 , la funzione $f(n)$ non sale al di sopra di $g(n)$ a meno di una costante moltiplicativa c_1 :

dunque nella funzione $g(n)$ non si indicano le costanti moltiplicative

Per esempio $f(n) = 2n^2 + 3n + 1$ è di ordine $\mathbf{O}(n^2)$
(contano solo i termini di grado massimo)



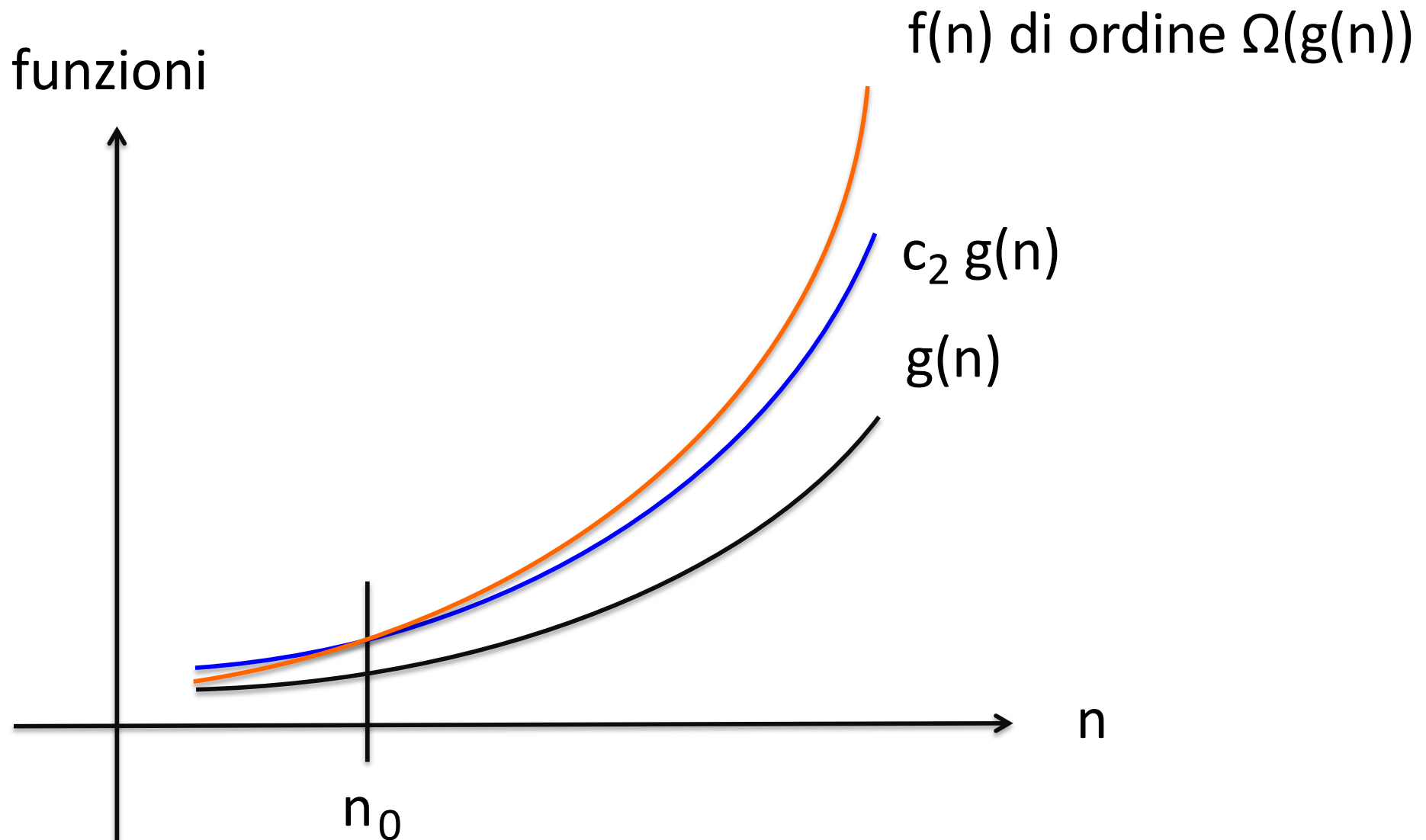
Ma la $2n^2 + 3n + 1$ è anche di ordine $\mathbf{O}(n^3)$!

La notazione $O(g(n))$ si impiega per indicare il tempo di un algoritmo:

- di cui **non** si conosce compiutamente il comportamento, ma che si sa che non può superare $g(n)$;
- oppure che **non** si comporta allo stesso modo per tutti gli insiemi di dati di dimensione n che gli si presentano, ma per alcuni richiede tempo proporzionale a $g(n)$, per altri meno.

Notazione Ω

$f(n)$ è di ordine $\Omega(g(n))$ se esistono due costanti positive c_2, n_0 , tali che $0 \leq c_2 g(n) \leq f(n)$ per ogni $n \geq n_0$.



Cioè, al crescere di n e a partire da un valore n_0 , la funzione $f(n)$ non scende al di sotto di $g(n)$ a meno di una costante moltiplicativa.

Anche qui contano solo i termini di grado massimo.

Per esempio $f(n) = 2n^2 - 3n + 1$ è di ordine $\Omega(n^2)$, ma anche $\Omega(n)$ ecc..

La notazione Ω si impiega per indicare il limite inferiore al tempo di soluzione di un problema.

Notare l'importante differenza nell'impiego dei due ordini O e Ω . Il primo è relativo al comportamento **di un particolare algoritmo** di soluzione, il secondo alla natura intrinseca del problema e si applica quindi a ***tutti i suoi algoritmi*** di soluzione

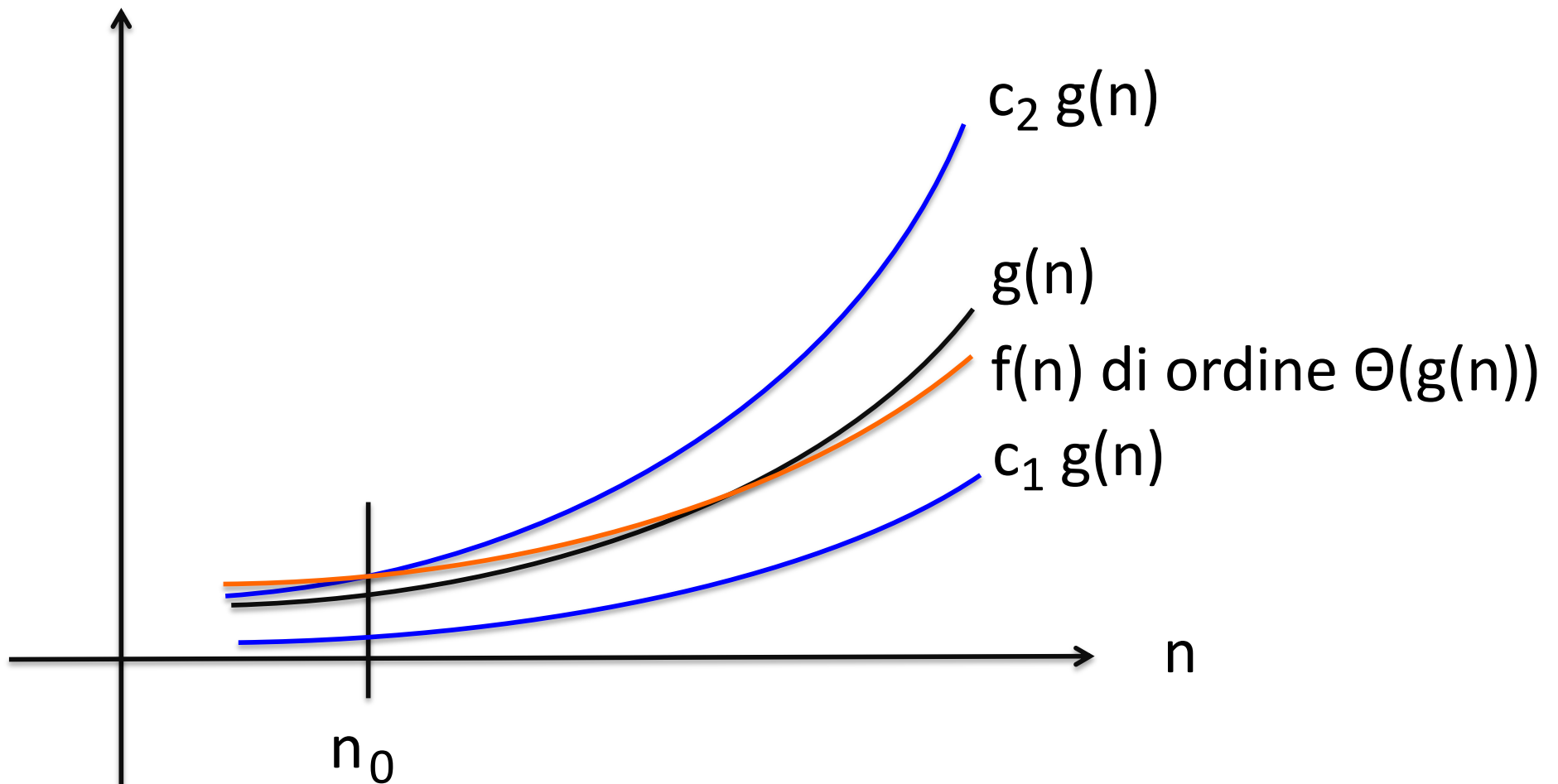
Notazione Θ

$f(n)$ è di ordine $\Theta(g(n))$ se esistono tre costanti positive c_1, c_2, n_0 , tali che $0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n)$ per ogni $n \geq n_0$.

Notazione Θ

$f(n)$ è di ordine $\Theta(g(n))$ se esistono tre costanti positive c_1, c_2, n_0 , tali che $0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n)$ per ogni $n \geq n_0$.

funzioni



Cioè, al crescere di n e a partire da un valore n_0 , le funzioni $f(n)$ e $g(n)$ hanno lo stesso andamento a meno di costanti moltiplicative.

Contano solo i termini di grado massimo.

Per esempio $f(n) = n^2 - 3n + 1$ è di ordine $\Theta(n^2)$, ma anche $f(n) = 3n^2 - 5$ è di ordine $\Theta(n^2)$ perché non si considerano le costanti moltiplicative.

La somma dei primi n interi $1 + 2 + \dots + n = n(n+1)/2$ è di ordine $\Theta(n^2)$.

La notazione Θ si impiega per indicare il tempo di un algoritmo di cui si conosce compiutamente il comportamento e che, a pari valore di n , si comporta allo stesso modo **per tutti gli insiemi** di dati di dimensione n che si presentano.